МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

ОТЧЁТ

по лабораторной работе №3

«Матрицы в 3D-графике»

по дисциплине «Практическая линейная алгебра»

Выполнил:

студент группы R3243 Осипов А. П.

Блок А: задания 1–3

Проверил:

Догадин Е. В.

Санкт-Петербург
2026

Содержание

Введение 3

Исходные данные 3

1 Задание 1. Отрисовка куба и другой фигуры 4

2 Задание 2. Масштабирование куба 6

3 Задание 3. Перенос и порядок преобразований 8

Заключение по заданиям 1–3 12

Приложение А. Код блока А 13

Введение

В отчёте представлен блок А лабораторной работы №3: отрисовка фигур в однородных координатах, масштабирование и перенос куба, сравнение порядка применения преобразований. Расчёты выполнены на Python с использованием NumPy и Matplotlib. Все преобразования координат реализованы умножением на матрицы.

Исходные данные

- Исходный куб из методички: координаты вершин по каждой оси равны -1 или 1, длина ребра — 2.
- Вершины тетраэдра, выбранные Альбертом: $A = (-2, -2, 0)$, $B = (4, -4, 0)$, $C = (0, 8, 0)$, $D = (1, 1, 10)$.
- Коэффициенты масштабирования, выбранные Альбертом для задания 2: $(s_{1x}, s_{1y}, s_{1z}) = (2, 4, 10)$ и $(s_{2x}, s_{2y}, s_{2z}) = (0.5, 1, 0.1)$.
- Векторы переноса, выбранные Альбертом для задания 3: $t_1 = (5, 5, 2)$ и $t_2 = (0, 3, 10)$.
- Для сравнения порядка преобразований используются $T = T_1$ и $S = S_1$.

1 Задание 1. Отрисовка куба и другой фигуры

Однородные координаты и устройство кода

Каждая вершина хранится в столбце матрицы `vertices` как $\widetilde{p} = (x, y, z, w)^T$. Для выбранных исходных фигур $w = 1$. Переход к декартовым координатам при $w \neq 0$ имеет вид

$$p = \begin{pmatrix} x/w \\ y/w \\ z/w \end{pmatrix}.$$

Например, $(4, 6, 2, 2)^T$ задаёт точку $(2, 3, 1)^T$. Строка программы

```
points = (vertices[:3, :] / vertices[3, :]).T
```

выполняет это деление для всех вершин. Транспонирование `.T` переводит матрицу размера $3 \times N$ в $N \times 3$: отрисовщику нужны вершины по строкам. Каждая строка `faces` задаёт индексы вершин одной грани в порядке обхода. Индексация начинается с нуля. `points[faces]` собирает координаты граней, а `Poly3DCollection` отображает их в пространстве. Использована ортографическая проекция `proj_type="ortho"`.

Исходный куб

Матрицы из Python-листинга методички:

$$V = \begin{pmatrix} -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix},$$

$$F = \begin{pmatrix} 0 & 1 & 5 & 4 \\ 1 & 2 & 6 & 5 \\ 2 & 3 & 7 & 6 \\ 3 & 0 & 4 & 7 \\ 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{pmatrix}.$$

У грани $[0, 1, 2, 3]$ все вершины имеют $z = -1$: это нижняя грань куба. Центр куба находится в начале координат, длина ребра равна 2.

Тетраэдр

Вместо куба заданы вершины $A = (-2, -2, 0)$, $B = (4, -4, 0)$, $C = (0, 8, 0)$, $D = (1, 1, 10)$:

$$V_{\text{tetra}} = \begin{pmatrix} -2 & 4 & 0 & 1 \\ -2 & -4 & 8 & 1 \\ 0 & 0 & 0 & 10 \\ 1 & 1 & 1 & 1 \end{pmatrix}, \qquad F_{\text{tetra}} = \begin{pmatrix} 0 & 2 & 1 \\ 0 & 1 & 3 \\ 1 & 2 & 3 \\ 2 & 0 & 3 \end{pmatrix}.$$

Первые три вершины лежат в плоскости $z = 0$, четвёртая — вне этой плоскости. Вместо шести четырёхугольных граней получены четыре треугольные. Пределы осей расширены под выбранные координаты; пропорции области построения согласованы с диапазонами осей, чтобы единичный масштаб по всем осям совпадал. Куб и тетраэдр показаны на рисунке 1.

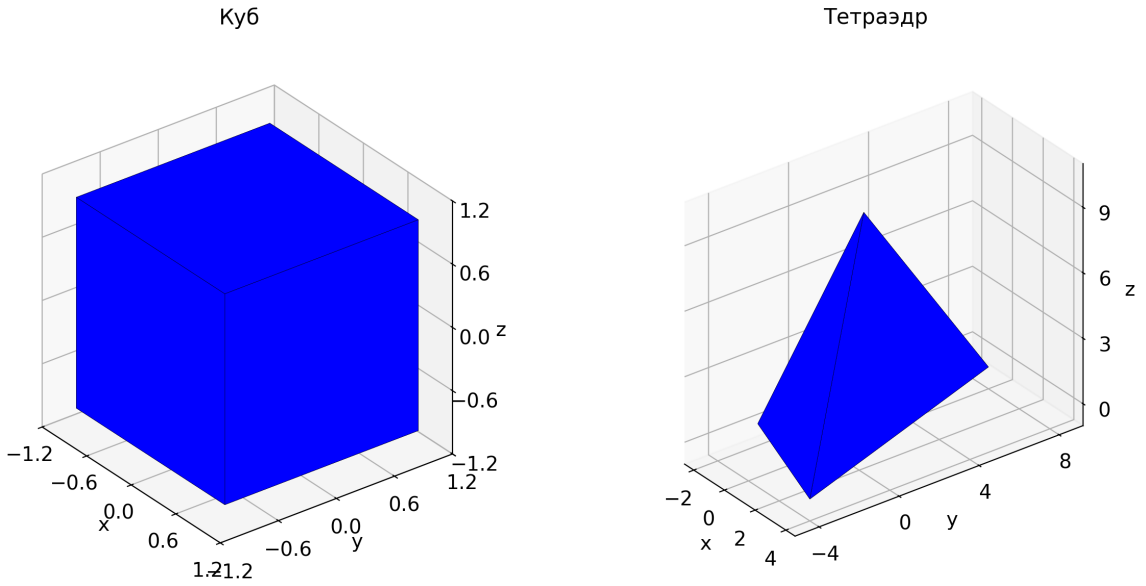


Рисунок 1 – Куб из методички и тетраэдр с выбранными вершинами

Вывод по заданию 1

Матрица 3×3 не может перенести начало координат в другую точку, поскольку произведение любой матрицы на нулевой вектор равно нулевому вектору. Четвёртая координата позволяет описать перенос матричным умножением; перед отрисовкой однородные координаты переводятся в декартовы делением на w.

2 Задание 2. Масштабирование куба

Общая структура матрицы

Масштабирование по координатным осям задаётся матрицей

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad S \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} s_x x \\ s_y y \\ s_z z \\ 1 \end{pmatrix}.$$

Матрицы масштаба отдельно по x, y, z получаются подстановкой $(s_x, 1, 1)$, $(1, s_y, 1)$, $(1, 1, s_z)$ соответственно. Все вершины преобразуются одновременно: $V' = SV$, в Python — `S @ vertices_cube`.

Выбранные матрицы и их композиция

$$S_1 = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad S_2 = \begin{pmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Сначала применяется S_1, затем S_2. Для векторов-столбцов последняя применяемая матрица стоит слева:

$$S_2(S_1 V) = (S_2 S_1)V, \qquad S_2 S_1 = \operatorname{diag}(0.5 \cdot 2,\ 1 \cdot 4,\ 0.1 \cdot 10,\ 1) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Отдельные матрицы S_1 и S_2 применяются к исходному кубу, а третий результат соответствует их последовательному применению.

Таблица 1 – Размеры куба после масштабирования

Преобразование	Размер по x	Размер по y	Размер по z
Исходный куб	2	2	2
S_1	4	8	20
S_2	1	2	0.2
$S_2 S_1$	2	8	2

Например, для вершины $p = (-1, -1, -1, 1)^T$:

$$S_1 p = (-2, -4, -10, 1)^T, \quad S_2 p = (-0.5, -1, -0.1, 1)^T, \quad S_2 S_1 p = (-1, -4, -1, 1)^T.$$

Результаты приведены на рисунке 2. У панелей разные пределы осей, поэтому абсолютные размеры нужно сравнивать по численным отметкам и таблице. Внутри каждой панели единичный масштаб одинаков по трём осям.

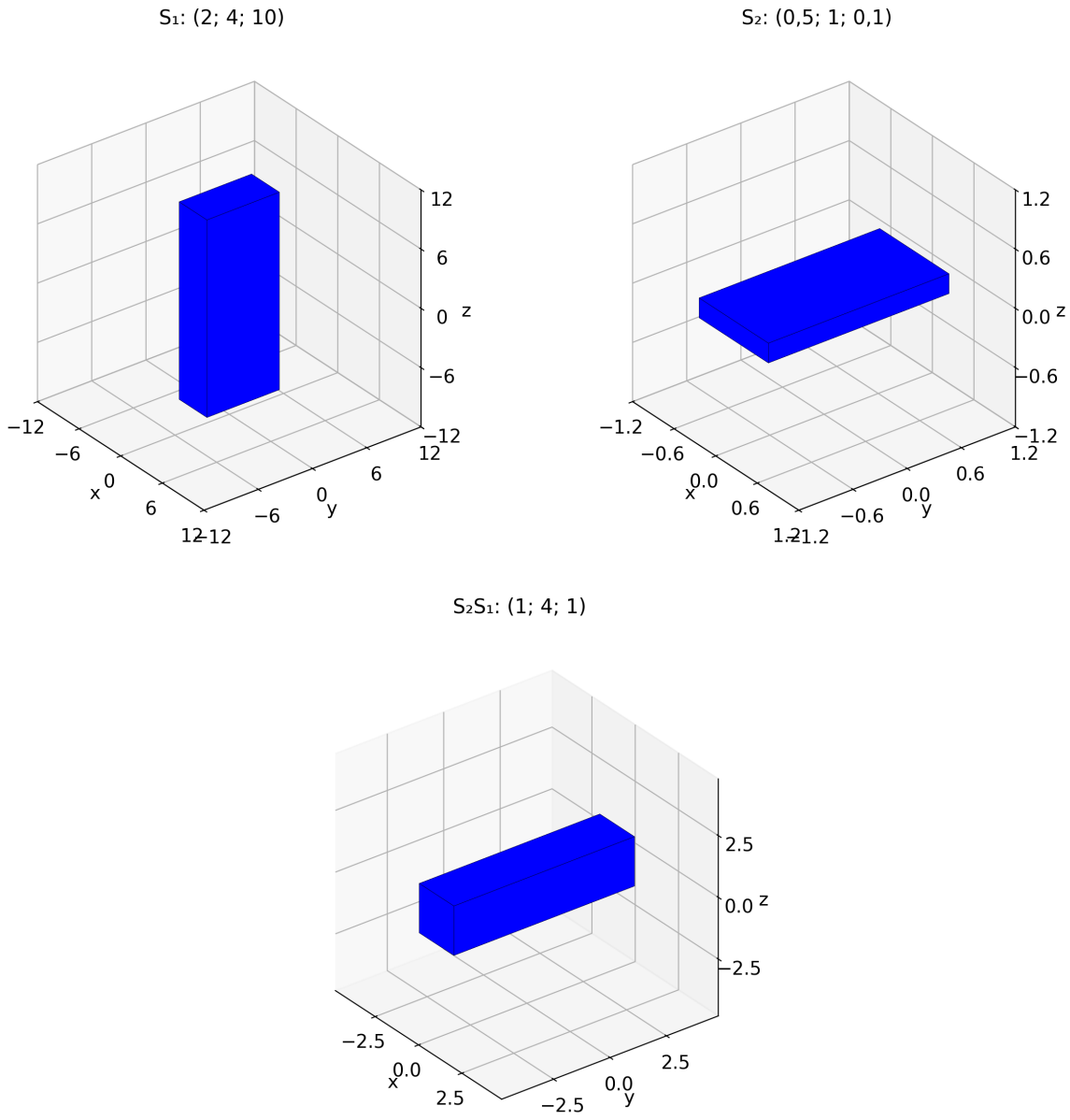


Рисунок 2 – Два масштаба и их композиция

Вывод по заданию 2

При последовательном масштабировании коэффициенты по каждой оси перемножаются. По оси x получаем $2 \cdot 0.5 = 1$, а по оси z — $10 \cdot 0.1 = 1$: второе преобразование компенсирует первое, поэтому размеры по этим осям возвращаются к исходным. По оси y произведение равно $4 \cdot 1 = 4$, поэтому растяжение в четыре раза сохраняется. Результат композиции — прямоугольный параллелепипед с размерами $(2, 8, 2)$.

3 Задание 3. Перенос и порядок преобразований

Общая структура матрицы переноса

$$T(a,b,c) = \begin{pmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x+a \\ y+b \\ z+c \\ 1 \end{pmatrix}.$$

Переносы отдельно по x, y, z задаются $T(a,0,0)$, $T(0,b,0)$, $T(0,0,c)$ соответственно. Последний столбец добавляет смещение к координатам.

Выбранные переносы и композиция

$$T_1 = \begin{pmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 5 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad T_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 10 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

При последовательном применении векторы переноса складываются:

$$T_2 T_1 = T(5+0,\ 5+3,\ 2+10) = \begin{pmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 8 \\ 0 & 0 & 1 & 12 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Центры образов куба равны $(5, 5, 2)$, $(0, 3, 10)$ и $(5, 8, 12)$ соответственно. Для исходной вершины $p = (-1, -1, -1, 1)^T$:

$$T_1 p = (4, 4, 1, 1)^T, \quad T_2 p = (-1, 2, 9, 1)^T, \quad T_2 T_1 p = (4, 7, 11, 1)^T.$$

Во всех трёх случаях размеры равны $(2, 2, 2)$. Результаты показаны на рисунке 3. Серым пунктиром обозначен исходный куб; у трёх панелей одинаковые пределы осей.

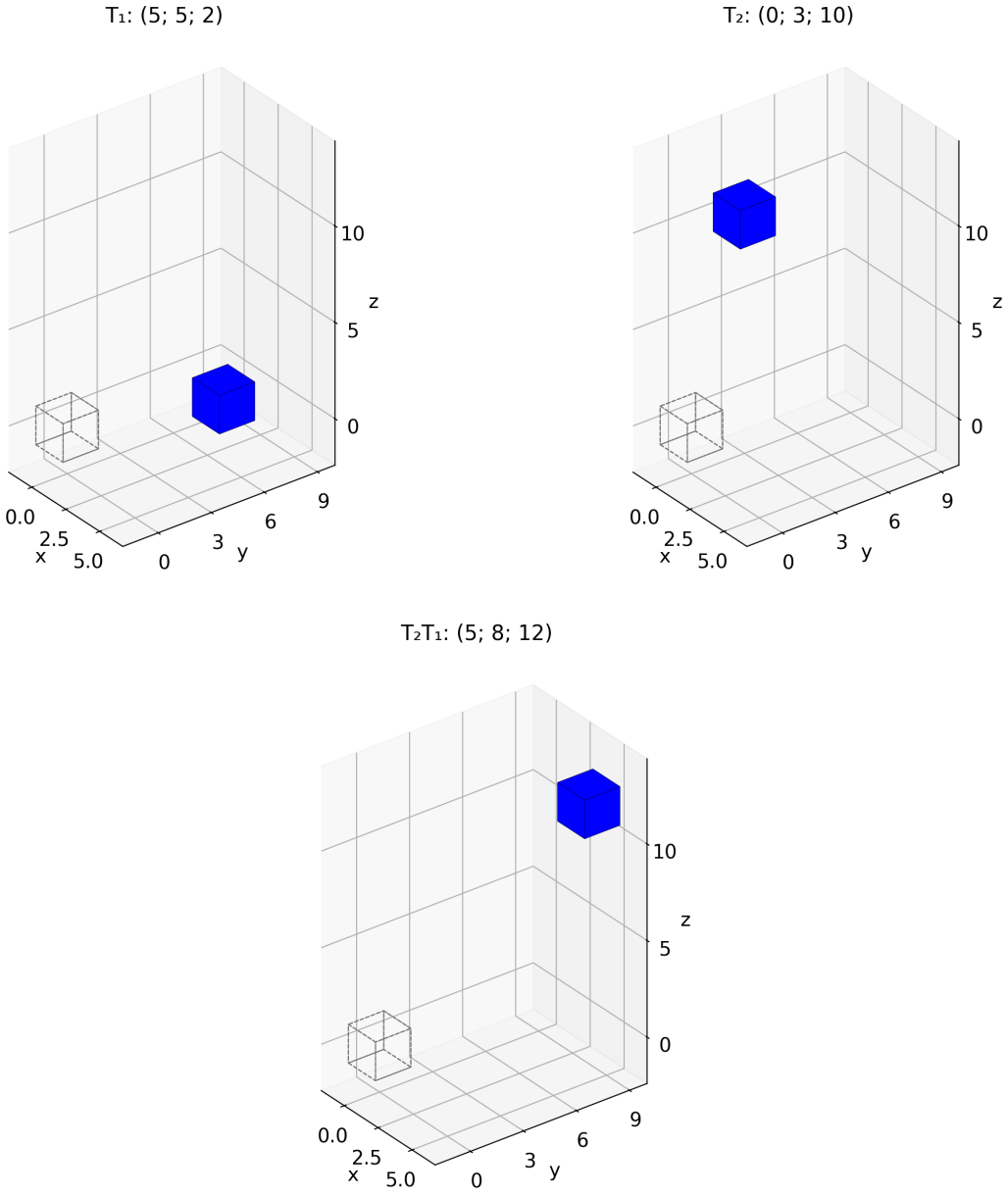


Рисунок 3 – Два переноса и их композиция; исходный куб показан контуром

Сравнение TS и ST

Для сравнения выбраны $T = T_1$ и $S = S_1$:

$$TS = \begin{pmatrix} 2 & 0 & 0 & 5 \\ 0 & 4 & 0 & 5 \\ 0 & 0 & 10 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad ST = \begin{pmatrix} 2 & 0 & 0 & 10 \\ 0 & 4 & 0 & 20 \\ 0 & 0 & 10 & 20 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

При TS точка сначала масштабируется, затем переносится:

$$(x, y, z) \longmapsto (2x + 5,\ 4y + 5,\ 10z + 2).$$

При ST точка сначала переносится, затем масштабируется:

$$(x, y, z) \longmapsto (2(x + 5),\ 4(y + 5),\ 10(z + 2)) = (2x + 10,\ 4y + 20,\ 10z + 20).$$

Например, $TSp = (3, 1, -8, 1)^T$, а $STp = (8, 16, 10, 1)^T$. Разность декартовых координат равна $(5, 15, 18)$ для каждой вершины.

Таблица 2 – Центры и размеры образов TS и ST

Преобразование	Центр образа	Размеры по x, y, z
TS	$(5, 5, 2)$	$(4, 8, 20)$
ST	$(10, 20, 20)$	$(4, 8, 20)$

Результаты неэквивалентны: матрицы отличаются последним столбцом и дают разные координаты образов. Сравнение приведено на рисунке 4.

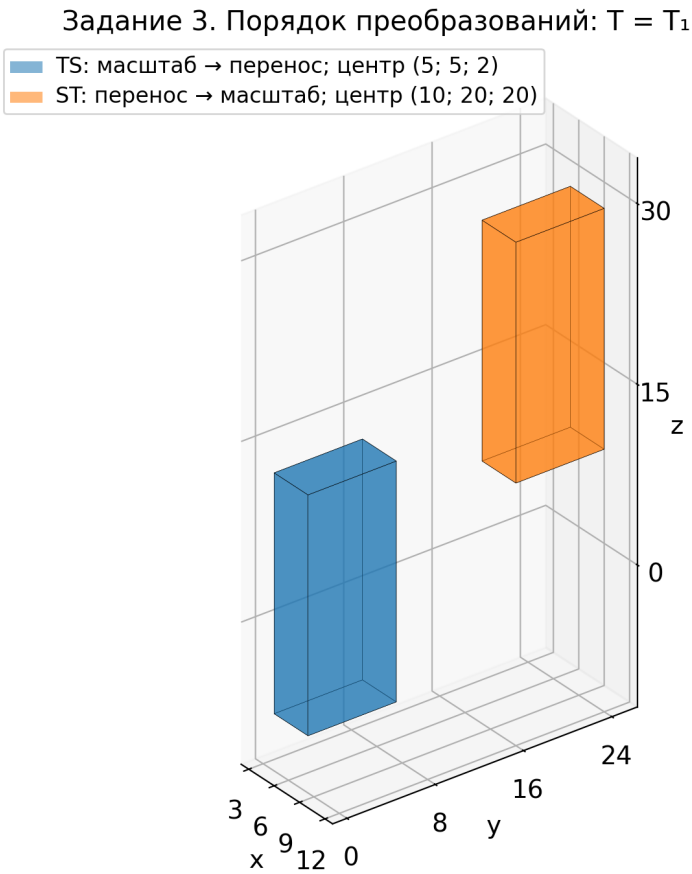


Рисунок 4 – Сравнение порядка применения масштабирования и переноса

Вывод по заданию 3

Перенос добавляет один и тот же вектор ко всем вершинам, поэтому разности их координат и размеры куба не меняются. Два последовательных переноса равносильны одному переносу на сумму векторов: $(5, 8, 12)$.
В общем случае умножение матриц некоммутативно: порядок преобразований может менять результат. Для выбранных матриц $TS \neq ST$. В TS масштаб применяется до переноса, а в ST масштабируется и добавленное смещение: $(5, 5, 2)$ превращается в $(2 \cdot 5, 4 \cdot 5, 10 \cdot 2) = (10, 20, 20)$. Поэтому положения образов различаются. Размеры в обоих случаях одинаковы, $(4, 8, 20)$, поскольку применяется один и тот же масштаб, а перенос размеры не меняет.
Определители произведений совпадают:

$$\det(TS) = \det(T)\det(S) = \det(S)\det(T) = \det(ST) = 1 \cdot 80 = 80.$$

Это следует из коммутативности умножения чисел и означает одинаковое изменение объёма — в 80 раз. Само по себе равенство определителей не гарантирует одинаковых размеров или положения фигур.

Заключение по заданиям 1–3

Однородные координаты позволяют задавать масштабирование и перенос матрицами одного размера 4×4. Композиция выбранных масштабов восстанавливает размеры по x и z, сохраняя растяжение по y. Два переноса складываются, а порядок переноса и масштабирования влияет на положение образа: в ST масштабируется и смещение. Равенство определителей TS и ST показывает одинаковое изменение объёма, но не равенство преобразований.

Приложение А. Код блока А

```
"""ЛР3, блок А: задания 1–3, фигуры, масштабирование и перенос куба."""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Patch
from matplotlib.ticker import MaxNLocator
from mpl_toolkits.mplot3d.art3d import Line3DCollection, Poly3DCollection


def draw_shape(ax, vertices, faces, color, *, alpha=1.0):
    """Нарисовать грани фигуры с вершинами в столбцах матрицы 4×N."""
    # Из (x, y, z, w) получаем декартовы координаты (x/w, y/w, z/w).
    points = (vertices[:3, :] / vertices[3, :]).T
    if color == "none":
        # Контур рисуем линиями: пустые грани Poly3DCollection поддерживаются не везде.
        contours = [points[np.r_[face, face[0]]] for face in faces]
        ax.add_collection3d(Line3DCollection(
            contours, colors="0.45", linewidths=0.8, linestyles="--"
        ))
        return
    ax.add_collection3d(
        Poly3DCollection(
            points[faces], facecolors=color, edgecolors="k", linewidths=0.2,
            alpha=alpha,
        )
    )


vertices_cube = np.array([
    [-1, 1, 1, -1, -1, 1, 1, -1],
    [-1, -1, 1, 1, -1, -1, 1, 1],
    [-1, -1, -1, -1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1, 1, 1, 1],
])

faces_cube = np.array([
    [0, 1, 5, 4],
    [1, 2, 6, 5],
    [2, 3, 7, 6],
    [3, 0, 4, 7],
    [0, 1, 2, 3],
    [4, 5, 6, 7],
])

vertices_tetra = np.array([
    [-2,  4, 0,  1],  # x
    [-2, -4, 8,  1],  # y
    [ 0,  0, 0, 10],  # z
    [ 1,  1, 1,  1],  # w
])

faces_tetra = np.array([
    [0, 2, 1],  # основание
    [0, 1, 3],  # боковые грани
    [1, 2, 3],
    [2, 0, 3],
])

```

```
# Задание 2. Масштабы, выбранные Альбертом; четвёртая координата не меняется.
S1 = np.diag([2.0, 4.0, 10.0, 1.0])
S2 = np.diag([0.5, 1.0, 0.1, 1.0])
# Для векторов-столбцов сначала применяется правая матрица.
S21 = S2 @ S1

# Задание 3. Переносы, выбранные Альбертом, стоят в последнем столбце.
T1 = np.array([
    [1, 0, 0, 5],
    [0, 1, 0, 5],
    [0, 0, 1, 2],
    [0, 0, 0, 1],
], dtype=float)
T2 = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 3],
    [0, 0, 1, 10],
    [0, 0, 0, 1],
], dtype=float)
T21 = T2 @ T1
# Сравниваем первые выбранные матрицы переноса и масштаба.
TS = T1 @ S1
ST = S1 @ T1


def set_scene_axes(ax, *shapes):
    """Охватить все переданные фигуры, сохранив одинаковую единицу по осям."""
    points = np.hstack([vertices[:3, :] / vertices[3, :] for vertices in shapes])
    lower = points.min(axis=1)
    upper = points.max(axis=1)
    padding = 0.1 * (upper - lower)
    lower, upper = lower - padding, upper + padding
    ax.set_xlim(lower[0], upper[0])
    ax.set_ylim(lower[1], upper[1])
    ax.set_zlim(lower[2], upper[2])
    ax.set_box_aspect(upper - lower, zoom=0.85)
    ax.view_init(azim=-37.5, elev=30)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
        axis.set_major_locator(MaxNLocator(nbins=4))


def plot_translation():
    """Переносы и их композиция: исходный куб показан контуром."""
    fig = plt.figure(figsize=(15, 5), layout="constrained")
    fig.suptitle("Задание 3. Перенос куба (исходный — контур)")
    cases = [
        ("T₁: (5; 5; 2)", T1),
        ("T₂: (0; 3; 10)", T2),
        ("T₂T₁: (5; 8; 12)", T21),
    ]
    images = [matrix @ vertices_cube for _, matrix in cases]
    for i, ((title, matrix), transformed) in enumerate(zip(cases, images), start=1):
        ax = fig.add_subplot(1, 3, i, projection="3d", proj_type="ortho")
        draw_shape(ax, vertices_cube, faces_cube, "none")
        draw_shape(ax, transformed, faces_cube, "blue")
        # Одинаковые пределы для трёх панелей делают сдвиги сравнимыми.
        set_scene_axes(ax, vertices_cube, *images)
        ax.set_title(title)
        print(f"\n{title}\n{matrix}")
        print("Центр куба:", transformed[:3, :].mean(axis=1))
```

```
    return fig


def plot_transform_order():
    """Сравнить перенос после масштаба и масштаб после переноса."""
    fig = plt.figure(figsize=(8, 7), layout="constrained")
    ax = fig.add_subplot(projection="3d", proj_type="ortho")
    vertices_ts = TS @ vertices_cube
    vertices_st = ST @ vertices_cube
    draw_shape(ax, vertices_ts, faces_cube, "tab:blue", alpha=0.55)
    draw_shape(ax, vertices_st, faces_cube, "tab:orange", alpha=0.55)
    set_scene_axes(ax, vertices_ts, vertices_st)
    ax.set_title("Задание 3. Порядок преобразований: T = T₁, S = S₁")
    ax.legend(handles=[
        Patch(facecolor="tab:blue", alpha=0.55,
              label="TS: масштаб → перенос; центр (5; 5; 2)"),
        Patch(facecolor="tab:orange", alpha=0.55,
              label="ST: перенос → масштаб; центр (10; 20; 20)"),
    ], loc="upper left", fontsize=9)
    for name, matrix, transformed in [("TS", TS, vertices_ts), ("ST", ST,
     ↪  vertices_st)]:
        print(f"\n{name}\n{matrix}")
        print("Центр куба:", transformed[:3, :].mean(axis=1))
        print("Размеры по x, y, z:", np.ptp(transformed[:3, :], axis=1))
    return fig


def plot_scaling():
    """Три результата: каждый отдельный масштаб и их последовательное применение."""
    fig = plt.figure(figsize=(15, 5), layout="constrained")
    fig.suptitle("Задание 2. Масштабирование куба")

    cases = [
        ("S₁: (2; 4; 10)", S1),
        ("S₂: (0,5; 1; 0,1)", S2),
        ("S₂S₁: (1; 4; 1)", S21),
    ]
    for i, (title, matrix) in enumerate(cases, start=1):
        transformed = matrix @ vertices_cube
        ax = fig.add_subplot(1, 3, i, projection="3d", proj_type="ortho")
        draw_shape(ax, transformed, faces_cube, "blue")

        # Свои пределы для каждого рисунка, но единичный масштаб по осям одинаков.
        # Куб центрирован в начале; все его вершины должны попасть в область.
        extent = np.max(np.abs(transformed[:3, :] / transformed[3, :]))
        limit = 1.15 * extent
        ax.set_xlim(-limit, limit)
        ax.set_ylim(-limit, limit)
        ax.set_zlim(-limit, limit)
        ticks = np.linspace(-extent, extent, 5)
        ax.set_xticks(ticks)
        ax.set_yticks(ticks)
        ax.set_zticks(ticks)
        ax.set_box_aspect([1, 1, 1], zoom=0.85)
        ax.view_init(azim=-37.5, elev=30)
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.set_zlabel("z")
        ax.set_title(title)

        print(f"\n{title}\n{matrix}")
        print("Размеры по x, y, z:", np.ptp(transformed[:3, :], axis=1))
    return fig
```

```
186
187
188 def main():
189     fig = plt.figure(figsize=(12, 6), layout="constrained")
190     ax = fig.add_subplot(1, 2, 1, projection="3d", proj_type="ortho")
191
192     draw_shape(ax, vertices_cube, faces_cube, "blue")
193
194     ax.set_box_aspect([1, 1, 1], zoom=0.85)
195     ax.set_xlim(-1, 1)
196     ax.set_ylim(-1, 1)
197     ax.set_zlim(-1, 1)
198     ax.view_init(azim=-37.5, elev=30)
199
200     ticks = np.linspace(-1, 1, 5)
201     ax.set_xticks(ticks)
202     ax.set_yticks(ticks)
203     ax.set_zticks(ticks)
204     ax.set_xlabel("x")
205     ax.set_ylabel("y")
206     ax.set_zlabel("z")
207     ax.set_title("Куб из методички")
208
209     ax_tetra = fig.add_subplot(1, 2, 2, projection="3d", proj_type="ortho")
210     draw_shape(ax_tetra, vertices_tetra, faces_tetra, "blue")
211
212     ax_tetra.set_xlim(-3, 5)
213     ax_tetra.set_ylim(-5, 9)
214     ax_tetra.set_zlim(-1, 11)
215     # Диапазоны осей 8, 14, 12: единичный масштаб одинаков по всем осям.
216     ax_tetra.set_box_aspect([8, 14, 12], zoom=0.85)
217     ax_tetra.view_init(azim=-37.5, elev=30)
218     ax_tetra.set_xlabel("x")
219     ax_tetra.set_ylabel("y")
220     ax_tetra.set_zlabel("z")
221     ax_tetra.set_title("Тетраэдр Альберта")
222
223     figures = {
224         "p01_figures": fig,
225         "p02_scaling": plot_scaling(),
226         "p03_translation": plot_translation(),
227         "p03_order": plot_transform_order(),
228     }
229     # Пути не зависят от рабочей папки: картинки черновика обновляются при запуске.
230     figures_dir = Path(__file__).resolve().parents[1] / "figures"
231     figures_dir.mkdir(parents=True, exist_ok=True)
232     for name, figure in figures.items():
233         path = figures_dir / f"{name}.png"
234         figure.savefig(path, dpi=200)
235         print(f"Сохранено: {path}")
236     plt.show()
237
238
239 if __name__ == "__main__":
240     main()
```

Листинг 1 – Отрисовка фигур, масштабирование и перенос куба